Лабораторная работа №2: Разработка API и контейнеризация модели

1. Общие сведения

- **Тема лабораторной работы:** «Разработка веб-интерфейса (API) и контейнеризация интеллектуальных приложений».
- **Трудоемкость:** 6 академических часов (3 лабораторные пары).
- **Цель лабораторной работы:** Освоить практические навыки проектирования легковесных, строго типизированных микросервисов для вывода моделей машинного обучения в промышленную эксплуатацию и научиться упаковывать их в изолированные, оптимизированные Docker-контейнеры.

Перечень формируемых компетенций и навыков

В результате выполнения лабораторной работы студент должен:

- **Знать:** архитектурные принципы клиент-серверного взаимодействия микросервисов по протоколу HTTP; паттерн проектирования Singleton применительно к жизненному циклу весов моделей в оперативной памяти; разницу между синхронной и асинхронной обработкой запросов в FastAPI (CPU-bound vs I/O-bound); концепцию слоистой файловой системы UnionFS и логику кэширования слоев в Docker; технологию многоэтапной сборки (Multi-stage builds).
- **Уметь:** проектировать контракты данных и схемы валидации входящих JSON-пакетов средствами библиотеки Pydantic; разрабатывать отказоустойчивые веб-эндпоинты POST-запросов для инференса моделей на FastAPI; описывать пошаговые манифесты сборки Dockerfile, исключающие нецелевые утилиты и кэш пакетных менеджеров; осуществлять проброс сетевых портов между изолированным контейнером и хост-системой.
- **Владеть:** навыками интерактивного тестирования веб-служб ИИ с помощью графических интерфейсов Swagger UI и Postman; методами минимизации и аудита размера готовых продакшен-образов (использование slim-дистрибутивов); навыками управления переменными окружения и конфигурациями веб-серверов Uvicorn внутри изолированных сред.

Используемое программное обеспечение и инструменты

Для успешного выполнения лабораторной работы студенту необходим персональный компьютер со следующим предустановленным стеком программного обеспечения:

- **Операционная система:** Linux (Ubuntu 22.04 LTS и новее), macOS или Windows 10/11 (с обязательным использованием подсистемы `WSL2`).
- **Среда разработки и версионирования:** Интерпретатор `Python 3.10` или `3.11`, система контроля версий `Git`.
- **Инструменты контейнеризации:** Движок `Docker Engine` и клиент `Docker CLI` (или пакет `Docker Desktop`), обеспечивающие изоляцию процессов на базе Linux-ядра.
- **Фреймворки веб-разработки:** Пакеты `fastapi` (для создания структуры микросервиса) и `uvicorn` (высокопроизводительный асинхронный ASGI веб-сервер).
- **Валидация данных и инференс:** `pydantic` (библиотека строгой типизации данных); фреймворк `onnxruntime` (движок для выполнения графа вычислений модели, экспортированной в Лабораторной работе №1).
- **Инструменты тестирования:** Встроенный в FastAPI интерфейс `Swagger UI` (доступный по эндпоинту `/docs`) или кроссплатформенное приложение `Postman` для конструирования HTTP-запросов.

2. Краткая теоретическая справка

2.1. Микросервисная архитектура и REST API для моделей машинного обучения

В промышленной индустрии разработки интеллектуальных систем обученная модель машинного обучения редко функционирует в качестве изолированного скрипта. Чтобы модель приносила бизнес-ценность, к ней должны беспрепятственно обращаться внешние потребители: мобильные клиенты, веб-интерфейсы, внутренние базы данных или CRM-системы. Для этого монолитные архитектуры разделяют на изолированные **микросервисы**, связующим звеном между которыми выступает **REST API** (*Representational State Transfer Application Programming Interface*), работающий поверх всемирного сетевого протокола **HTTP**.

Сущность клиент-серверного взаимодействия ИИ-систем по протоколу HTTP

В контексте MLOps взаимодействие систем строится на базе классической **клиент-серверной архитектуры** и подчиняется строго детерминированному циклу **«Запрос – Ответ» (Request – Response)**. Клиентское приложение (например, фронтенд интернет-

магазина) ничего не знает о внутренней математике нейросети или слоях графа вычислений. Его задача – собрать сырые признаки (текст, картинку, числовой вектор), упаковать их по правилам сетевого протокола и отправить HTTP-запрос на определенный сетевой адрес сервера (эндпоинт).

Развертывание моделей строго на стороне выделенного сервера (а не внутри кода клиента) обусловлено тремя инженерными причинами:

1. **Вычислительные лимиты хоста:** Современные глубокие сети требуют гигабайты оперативной памяти (RAM/VRAM) и мощные GPU-ускорители, которых нет на устройствах обычных пользователей.
2. **Защита интеллектуальной собственности:** Веса модели представляют собой коммерческую тайну компании. Модель, зашитая на стороне клиента, уязвима для декомпиляции и кражи алгоритма. Серверный контур полностью изолирован от прямого доступа злоумышленников.
3. **Бесшовное обновление (Continuous Delivery):** В случае деградации качества модели или изменения распределения данных (Data Drift) инженер может мгновенно подменить веса в Model Registry на сервере. При этом пользователям не нужно скачивать обновления для своих приложений из App Store или Google Play.

Анатомия POST-запроса и ответа (структура JSON-пакетов при инференсе)

Для передачи сложных структурированных объектов по сети REST API использует текстовый формат обмена данными **JSON** (*JavaScript Object Notation*), представляющий собой словарь формата «ключ: значение». В задачах инференса (генерации предсказаний моделью) сетевой обмен имеет строгую анатомию.

- **HTTP-запрос (Request):** Для инференса всегда используется метод `POST`. Этот метод, в отличие от метода `GET`, не отображает передаваемые параметры в открытом виде в адресной строке браузера, а надежно инкапсулирует (прячет) их внутрь тела запроса (*Request Body*). Это защищает персональные данные пользователей и позволяет передавать массивы признаков любого объема.*Пример тела JSON-запроса от клиента:*

```
{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}
```

- **HTTP-ответ (Response):** Сервер принимает входящий байтовый поток, парсит JSON, передает извлеченные числа в модель, формирует предсказание и возвращает клиенту пакет ответа. Ответ содержит **Статус-код HTTP** (например, `200 OK` при успешном вычислении, `422 Unprocessable Entity` при ошибке валидации данных

или `500 Internal Server Error` при падении рантайма модели) и тело ответа с результатом работы алгоритма.*Пример тела JSON-ответа от сервера:*

```
{
  "predicted_class": 0,
  "probabilities": [0.984, 0.011, 0.005],
  "model_version": "iris_onnx_v1.0"
}
```

Паттерн проектирования Stateless (независимость и масштабируемость контейнеров)

Ключевым архитектурным требованием к промышленным ИИ-микросервисам является соблюдение паттерна **Stateless (без сохранения состояния)**. Это означает, что веб-сервер, обрабатывающий запросы, не должен сохранять в своей локальной памяти историю предыдущих кликов пользователя, контекст прошлых запросов или состояние сессии. Каждый входящий HTTP-запрос обязан быть абсолютно изолированным, независимым и самодостаточным – то есть содержать в своем теле *все* необходимые признаки, которые требуются модели для выдачи мгновенного предсказания «здесь и сейчас».

Соблюдение Stateless-архитектуры критически важно для **горизонтального масштабирования** системы. Если посещаемость вашего интеллектуального приложения резко вырастет, системные администраторы поднимут в облаке или кластере Kubernetes десять абсолютно одинаковых Docker-контейнеров с вашей моделью и поставят перед ними балансировщик нагрузки (Load Balancer). Балансировщик будет распределять запросы пользователей по контейнерам случайным образом. Так как сервисы Stateless, любой из десяти контейнеров сможет принять очередной запрос, корректно передать его модели и вернуть точный ответ, гарантируя отказоустойчивость всей ИТ-инфраструктуры под высокой нагрузкой.

2.2. Управление жизненным циклом модели в FastAPI

При проектировании веб-сервисов для машинного обучения стандартные подходы веб-разработки часто наталкиваются на жесткие ограничения вычислительной математики. Игнорирование специфики ИИ-моделей на этапе написания серверного кода приводит к критическим отказам инфраструктуры.

Критическая проблема повторной загрузки весов

Типичная архитектурная ошибка начинающих разработчиков – инициализация и загрузка модели с диска непосредственно внутри функции обработки входящего запроса (эндпоинта).

Файлы весов современных нейронных сетей или ансамблей алгоритмов имеют размер от сотен мегабайт до десятков гигабайт. Процесс считывания бинарного графа с диска, парсинг его структуры и выделение памяти в оперативной памяти (RAM) или

видеопамяти (VRAM) – это крайне тяжелая операция, занимающая от нескольких секунд до минут.

Если модель инициализируется внутри эндпоинта, возникают две катастрофические проблемы:

1. **Критический рост задержки (Latency Overhead):** Каждый пользователь при отправке запроса будет ждать завершения дисковых операций чтения. Метрика Latency вырастет до неприемлемых значений.
2. **Переполнение памяти (Out of Memory – OOM):** При одновременном поступлении нескольких десятков параллельных HTTP-запросов сервер попытается создать соответствующее количество копий модели в оперативной памяти. Ресурсы RAM/VRAM мгновенно исчерпаются, и операционная система принудительно уничтожит процесс веб-сервера с помощью механизма `OOM Killer`.

Паттерн Singleton (Одиночка) и механизм Lifespan

Для устранения этих рисков в MLOps применяется паттерн проектирования **Singleton (Одиночка)**. Согласно этому паттерну, веса модели должны загружаться в оперативную память **строго один раз – в момент инициализации и старта веб-сервера**, до того как система начнет принимать пользовательский трафик. Модель должна постоянно удерживаться в RAM в качестве глобального, неизменяемого и доступного только для чтения объекта. Все входящие HTTP-запросы должны мгновенно обращаться к этому уже «прогретому» объекту.

Во фреймворке FastAPI этот паттерн реализуется через современный механизм событий жизненного цикла – менеджер контекста **`lifespan`**. Он гарантирует строгое разделение этапов работы приложения:

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
import onnxruntime as ort

# Глобальная переменная для хранения сессии инференса
onnx_session = None

@asynccontextmanager
async def lifespan(app: FastAPI):
    global onnx_session
    # [Блок STARTUP]: Выполняется строго ОДИН РАЗ при запуске сервера
    print("Инициализация сервера: загрузка графа ONNX в оперативную память...")
    onnx_session = ort.InferenceSession("iris_model.onnx")
    yield
    # [Блок SHUTDOWN]: Выполняется строго ОДИН РАЗ при остановке сервера
    print("Остановка сервера: освобождение ресурсов памяти...")
    del onnx_session

app = FastAPI(lifespan=lifespan)
```

Специфика параллельной обработки: `def` против `async def`

FastAPI – асинхронный фреймворк, работающий на базе цикла событий (**Event Loop**). Однопоточный Event Loop эффективно переключается между задачами сетевого ввода-вывода (**I/O-bound**), такими как ожидание ответа от базы данных или скачивание файла. Пока один запрос ждет сеть, Event Loop обслуживает других пользователей.

Однако инференс ML-модели (вызов метода `.run()` в ONNX Runtime или `.predict()` в Scikit-learn) – это **CPU-bound задача** (интенсивные математические вычисления и перемножение матриц на уровне процессора), которая не содержит асинхронных пауз ожидания (`await`).

Объявление эндпоинта инференса и выбор ключевых слов имеют решающее значение:

1. **Неверный подход – Использование `async def` для математики:** Если объявить эндпоинт инференса как `async def predict(...)`, то тяжелая математическая операция захватит единственный поток Event Loop. Весь цикл событий намертво замрет на время вычисления. Сервер FastAPI «ослепнет»: он перестанет принимать новые входящие TCP-пакеты, не сможет отвечать другим пользователям и завалит технический мониторинг.
2. **Верный подход – Использование стандартного `def`:** Если объявить эндпоинт как стандартную синхронную функцию `def predict(...)`, FastAPI автоматически распознает, что внутри будет выполняться блокирующий CPU-bound код. Сервер принудительно вынесет выполнение этой функции из основного потока Event Loop в отдельный фоновый поток из встроенного пула потоков (**Asynchronous Thread Pool**). Основной поток Event Loop останется абсолютно свободным, продолжит принимать тысячи запросов в секунду и складывать их в системную очередь, пока ядра процессора параллельно вычисляют предсказания модели в фоновых потоках.

2.3. Строгая типизация и самодокументирование веб-служб

В промышленной разработке микросервисов взаимодействие между различными командами (например, инженерами машинного обучения и разработчиками фронтенда или мобильных приложений) часто упирается в человеческий фактор. Командам необходим строгий, актуальный и прозрачный контракт обмена данными. Фреймворк FastAPI решает эту задачу на стыке двух технологий: библиотеки **Pydantic** и стандарта **OpenAPI** со встроенным визуализатором **Swagger UI**.

Валидация входящего трафика средствами Pydantic

Библиотека **Pydantic** является неотъемлемой частью FastAPI и отвечает за парсинг, строгую типизацию и валидацию данных прямо на входе в веб-сервер. В машинном обучении это служит важнейшим контуром безопасности. Входной граф вычислений нейросети или классического алгоритма (например, в ONNX Runtime) жестко типизирован и ожидает строго определенную размерность матриц. Если передать туда некорректные данные (например, строковое значение вместо числа или вектор неверной длины), низкоуровневая C++ библиотека упадет с критической ошибкой сегментации памяти (`Segmentation Fault`), что приведет к мгновенному падению всего веб-сервера.

Описывая структуру входящего JSON-пакета, разработчик создает класс, наследуемый от `pydantic.BaseModel`. Инструмент использует механизм декларативного описания подсказок типов (**Type Hinting**) языка Python:

```
from pydantic import BaseModel, Field

class IrisInputData(BaseModel):
    # Field позволяет задать валидацию, текстовое описание и пример для документации
    sepal_length: float = Field(..., gt=0, description="Длина чашелистика в см", example=5.1)
    sepal_width: float = Field(..., gt=0, description="Ширина чашелистика в см", example=3.5)
    petal_length: float = Field(..., gt=0, description="Длина лепестка в см", example=1.4)
    petal_width: float = Field(..., gt=0, description="Ширина лепестка в см", example=0.2)
```

Как только HTTP-запрос прилетает на сервер, Pydantic перехватывает его и выполняет тотальную валидацию:

- Если клиент прислал корректный JSON, Pydantic автоматически преобразует типы (например, строку `"5.1"` сконвертирует в число `5.1` типа `float`) и соберет из них чистый Python-объект.
- Если в запросе отсутствует обязательное поле, нарушено условие `gt=0` (значение должно быть строго больше нуля) или вместо чисел переданы буквы, Pydantic мгновенно заблокирует выполнение кода эндпоинта. Он вернет клиенту стандартизированный ответ со статус-кодом `422 Unprocessable Entity` и подробным JSON-описанием того, в каком именно поле и какая конкретно ошибка была допущена. Модель при этом полностью защищена от «грязных» данных.

Концепция самодокументирования кода: OpenAPI и Swagger UI

Благодаря тому, что контракты данных описаны через стандартные Type Hints и Pydantic-модели, FastAPI на лету, в автоматическом режиме генерирует полную схему вашего веб-сервиса по международному открытому стандарту **OpenAPI** (ранее известному

как Swagger Specification). Это декларативный JSON-чертеж, который детально описывает все эндпоинты, типы запросов, обязательные поля, форматы ответов и возможные коды ошибок.

На основе этой OpenAPI-схемы FastAPI из коробки разворачивает графический интерактивный интерфейс – **Swagger UI**. Он доступен по умолчанию на любом запущенном сервере FastAPI по адресу: `http://127.0.0.1:8000/docs`

Swagger UI решает проблему устаревания текстовой документации (в Confluence или PDF), так как генерируется динамически из актуального исходного кода прямо в момент старта сервера.

Роль Swagger UI в командной интеграции:

1. **Интерактивная песочница:** Любой разработчик смежной команды или тестировщик может открыть эту веб-страницу, нажать кнопку *«Try it out»*, прямо в браузере отредактировать автоматически сгенерированный тестовый JSON-шаблон и нажать кнопку *«Execute»*. Сервер выполнит реальный инференс модели и отобразит точный HTTP-ответ с временем задержки. Для тестирования модели больше не нужны сторонние утилиты (Postman) или локально настроенное Python-окружение.
2. **Автоматическая генерация клиентов:** Поскольку Swagger UI предоставляет чистую OpenAPI-спецификацию в формате JSON (по адресу `/openapi.json`), разработчики мобильных приложений или бэкенда на других языках могут использовать утилиты автоматической кодогенерации (например, *OpenAPI Generator*). Они могут в один клик скомпилировать готовые библиотеки сетевых запросов к вашей модели на TypeScript, Swift, Java или C++, полностью исключая ручные ошибки при интеграции интеллектуального микросервиса.

2.4. Технологии контейнеризации интеллектуальных приложений на базе Docker

Успешно разработанный и протестированный локально микросервис на FastAPI необходимо бесшовно доставить в промышленную ИТ-инфраструктуру предприятия. Главным барьером на этом этапе является «кризис окружения»: различия в версиях интерпретатора Python, операционной системы, системных библиотек (таких как `glibc`) и вычислительных пакетов на машине разработчика и на продакшен-сервере приводят к критическим сбоям. Решением этой проблемы является технология контейнеризации **Docker**.

Контейнеризация против виртуализации

В отличие от классической виртуализации (виртуальных машин KVM, VMware, VirtualBox), где внутри основного сервера разворачивается тяжелый гипервизор и полноценная гостевая операционная система, пожирающая гигабайты RAM на собственное существование, Docker работает нативно.

Контейнеры Docker **не содержат внутри себя гостевую ОС**. Все запущенные контейнеры используют **единое, общее ядро** операционной системы хост-сервера. Изоляция процессов, файловой системы и сетевого стека внутри контейнера достигается за счет встроенных механизмов ядра Linux: пространств имен (**Namespaces**) и контрольных групп (**Cgroups**), которые жестко лимитируют выделяемые ресурсы CPU и RAM. Благодаря этому Docker-контейнер представляет собой обычный изолированный процесс, который запускается за доли секунды, расходует ноль мегабайт памяти на эмуляцию железа и выполняет математические вычисления с нативной скоростью процессора.

Слоистая архитектура UnionFS и правила оптимизации кэша

В основе образов Docker лежит слоистая файловая система (**Union File System – UnionFS**). Docker-образ (*Docker Image*) представляет собой пирог из последовательных, неизменяемых слоев, доступных только для чтения (`read-only`). Каждая значимая директива в конфигурационном файле **Dockerfile** (`FROM`, `RUN`, `COPY`) создает новый, фиксированный слой, содержащий только изменения (дельту) относительно предыдущего.

При сборке образа (`docker build`) движок Docker использует агрессивное **кэширование слоев**. Если строка в Dockerfile и файлы, которые она затрагивает, не изменились, Docker не выполняет команду заново, а мгновенно берет готовый слой из локального кэша.

Для эффективной сборки ML-приложений инженер обязан соблюдать строго определенный порядок команд, переходя от редко меняющихся компонентов к часто меняющимся:

```
# 1. Редко меняется: Базовая ОС
FROM python:3.10-slim
WORKDIR /app

# 2. Меняется редко: Список библиотек. Копируем ТОЛЬКО requirements.txt
COPY requirements.txt .
# Запекаем тяжелый слой pip install. Флаг --no-cache-dir обязателен,
# чтобы бинарные архивы пакетов не оставались внутри слоя и не раздували образ.
RUN pip install --no-cache-dir -r requirements.txt

# 3. Меняется ПОСТОЯННО: Исходный код инференса и веса модели
COPY inference_onnx.py .
COPY iris_model.onnx .

CMD ["python", "inference_onnx.py"]
```

Если бы мы написали директиву `COPY . .` (скопировать всё разом) в самом начале, то при малейшем изменении хотя бы одного символа в коде скрипта инференса Docker посчитал бы кэш этой строки невалидным, сбросил бы все последующие слои и заново скачивал и устанавливал тяжелые пакеты `numpy` и `onnxruntime` из интернета при каждой сборке.

Технология многоэтапной сборки (Multi-stage builds)

Специфика ИИ-приложений заключается в том, что для сборки, компиляции некоторых бинарных зависимостей или кастомных операторов ONNX требуются тяжелые системные утилиты: компиляторы `gcc`, `g++`, `make` и заголовочные файлы `python3-dev`. Они весят сотни мегабайт, повышают вес образа и увеличивают поверхность атаки для хакеров. Но в готовом продакшен-контейнере эти компиляторы не нужны – там нужен только результат их работы (скомпилированные `.whl` файлы или бинарные библиотеки).

Для решения этой проблемы применяется паттерн **Multi-stage builds (Многоэтапная сборка)**. В рамках одного Dockerfile инженер описывает несколько изолированных этапов сборки, используя несколько директив `FROM`:

1. **Этап сборщика (Builder Stage):** Базируется на стандартном образе, куда принудительно устанавливаются все необходимые компиляторы. Внутри этого изолированного слоя скачиваются, компилируются и собираются все пакеты. [4]
2. **Финальный этап (Production Stage):** Инициализирует кристально чистый, легковесный базовый образ (например, `python-slim`). С помощью специальной директивы `COPY --from=...` инженер копирует из первого этапа *только* готовые скомпилированные результаты и бинарные библиотеки, полностью оставляя все тяжелые компиляторы, мусор сборки и временные файлы за бортом.

Использование технологии Multi-stage сборки позволяет сократить размер продакшен-образа в несколько раз, гарантируя высокую скорость развертывания (Continuous Deployment) и максимальную безопасность интеллектуального микросервиса.

3. Задания и ход выполнения работы

Задание 1: Разработка REST API на базе фреймворка FastAPI

Этап 1.1. Проектирование контрактов данных (схем входящего запроса и исходящего ответа) с помощью классов BaseModel и Field библиотеки Pydantic. Настройка строгой валидации признаков.

Для обеспечения безопасности вычислительного графа и интеграции микросервиса в ИТ-инфраструктуру, необходимо жестко описать контракты обмена данными средствами библиотеки Pydantic.

1. Откройте вашу среду разработки (IDE) в корневой директории проекта (из Лабораторной работы №1) и создайте новый пустой файл с именем `main.py`.

2. Пропишите в самом начале созданного файла импорт необходимых инструментов для валидации данных и типизации:

```
from typing import List
from pydantic import BaseModel, Field
```

3. Опишите класс входящего запроса, унаследовав его от базовой модели Pydantic, и зафиксируйте в нем четыре обязательных признака модели со строгим указанием типов:

```
class IrisRequestData(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float
```

4. Модернизируйте объявление переменных внутри класса IrisRequestData, добавив функцию Field. Настройте обязательность полей через оператор ... и заблокируйте передачу некорректных отрицательных значений с помощью флага gt=0:

```
class IrisRequestData(BaseModel):
    sepal_length: float = Field(..., gt=0, description="Длина чашелистика в
см", example=5.1)
    sepal_width: float = Field(..., gt=0, description="Ширина чашелистика в
см", example=3.5)
    petal_length: float = Field(..., gt=0, description="Длина лепестка в см",
example=1.4)
    petal_width: float = Field(..., gt=0, description="Ширина лепестка в см",
example=0.2)
```

5. Ниже по коду спроектируйте класс исходящего ответа IrisResponseData, который будет возвращать вызывающей стороне индекс класса, вектор уверенности и метаданные версии:

```
class IrisResponseData(BaseModel):
    predicted_class: int = Field(..., description="Индекс предсказанного
класса (0, 1 или 2)")
    probabilities: List[float] = Field(..., description="Вектор вероятностей
распределения по классам")
    model_status: str = Field(..., description="Системный статус и
идентификатор версии инференса")
```

6. Сохраните изменения в файле main.py.

Этап 1.2. Настройка управления жизненным циклом веб-приложения с использованием менеджера контекста lifespan. Реализация паттерна Singleton для однократной загрузки графа вычислений ONNX Runtime в оперативную память при старте сервера.

Для исключения накладных расходов памяти и предотвращения повторной загрузки весов модели с диска при каждом запросе, необходимо реализовать паттерн Singleton с использованием менеджера контекста `lifespan`.

1. Добавьте в блок импортов файла `main.py` контекстный менеджер из стандартной библиотеки Python и движок исполнения ONNX Runtime:

```
from contextlib import asynccontextmanager
import onnxruntime as ort
```

2. Объявите в глобальной области видимости переменную onnx_session со значением по умолчанию None. Она будет служить единой точкой доступа к прогретой модели в оперативной памяти:

```
onnx_session = None
```

3. Ниже классов Pydantic опишите асинхронную функцию управления жизненным циклом приложения lifespan с декоратором @asynccontextmanager. Запрограммируйте в ней однократную инициализацию сессии инференса на этапе запуска сервера:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    global onnx_session
    print("Инициализация сервера: загрузка графа ONNX в оперативную
память...")
    onnx_session = ort.InferenceSession("iris_model.onnx")
    yield
    print("Остановка сервера: освобождение ресурсов памяти...")
    del onnx_session
```

(Примечание: убедитесь, что экспортированный в первой лабораторной работе файл `iris_model.onnx` физически находится в корневом каталоге проекта рядом со скриптом `main.py`)

4. Инициализируйте основной экземпляр приложения FastAPI, принудительно передав созданный контекстный менеджер в именованный параметр lifespan:

```
app = FastAPI(lifespan=lifespan)
```

(Примечание: не забудьте добавить `from fastapi import FastAPI` в самый верхний блок импортов вашего файла)

5. Сохраните изменения в файле main.py.

Этап 1.3. Создание синхронного эндпоинта (POST /v1/predict) для инференса модели с применением стандартного объявления функции def для автоматического выноса CPU-bound вычислений в фоновый пул потоков.

Для предотвращения блокировок основного асинхронного потока сервера (Event Loop) при выполнении тяжелых матричных вычислений, необходимо создать синхронный эндпоинт, который FastAPI автоматически вынесет во внешний пул фоновых потоков.

1. Импортируйте библиотеку `numpy` в верхнюю часть файла `main.py` для корректного формирования входных массивов (тензоров) в ONNX Runtime:

```
import numpy as np
```

2. Опишите в конце файла main.py функцию-эндпоинт с использованием стандартного (синхронного) объявления def, привязав её к сетевому адресу /v1/predict с помощью декоратора @app.post:

```
@app.post("/v1/predict", response_model=IrisResponseData)
def predict(data: IrisRequestData):
    pass
```

(Примечание: явное указание `response_model` гарантирует, что FastAPI автоматически отвалидирует и отфильтрует структуру выходных данных на основе созданного ранее класса Pydantic)

3. Замените заглушку pass внутри функции predict логикой извлечения признаков из входящего JSON-пакета Pydantic и их упаковки в двумерный массив NumPy строго с типом float32:

```
    input_array = np.array([[
        data.sepal_length,
        data.sepal_width,
        data.petal_length,
        data.petal_width
    ]], dtype=np.float32)
```

4. Реализуйте обращение к глобальной сессии инференса onnx_session, выполнив прямой проход по графу вычислений (DAG) модели, и извлеките предсказанный класс и вектор вероятностей:

```
    input_name = onnx_session.get_inputs()[0].name
    raw_predictions = onnx_session.run(None, {input_name: input_array})

    # ONNX Runtime возвращает список тензоров. Извлекаем нужные элементы для батча размера 1
    predicted_class = int(raw_predictions[0][0])
    probabilities = raw_predictions[1][0].tolist()  # Конвертируем массив numpy в стандартный список Python
```

5. Сформируйте и верните словарь с финальным результатом, структура которого строго соответствует контракту IrisResponseData:

```
    return {
        "predicted_class": predicted_class,
        "probabilities": probabilities,
        "model_status": "iris_onnx_v1.0"
    }
```

6. Сохраните изменения в файле main.py. Убедитесь, что итоговая структура вашего скрипта включает в себя импорты, Pydantic-схемы, блок lifespan и созданный эндпоинт predict.

Задание 2: Локальное тестирование и самодокументирование сервиса

Этап 2.1. Запуск локального ASGI-сервера Uvicorn внутри виртуального окружения, настройка прослушивания сетевых портов.

Для запуска и проверки разработанного микросервиса необходимо использовать высокопроизводительный ASGI-сервер Uvicorn, который будет слушать сетевой порт и передавать входящий HTTP-трафик в приложение FastAPI.

1. Откройте терминал, убедитесь, что вы находитесь в корневой директории проекта, и активируйте виртуальное окружение, если оно было деактивировано:
   - **Linux / macOS:**
   ```
   source venv/bin/activate
   ```
   - **Windows:**
   ```
   venv\Scripts\activate.bat
   ```
2. Запустите веб-сервер Uvicorn, передав ему в качестве аргумента путь к вашему приложению в формате `имя_файла:имя_переменной_FastAPI`, а также настройте локальный сетевой адрес и порт:

```
uvicorn main:app --host 127.0.0.1 --port 8000 --reload
```

(Примечание: флаг `--reload` активирует режим автоматического перезапуска сервера при изменении исходного кода, что крайне удобно на этапе разработки и отладки).

3. Внимательно изучите текстовый вывод в консоли терминала. Убедитесь, что сервер успешно запустился и вы видите следующие информационные логи:

```
INFO:     Started server process
INFO:     Waiting for application startup.
Инициализация сервера: загрузка графа ONNX в оперативную память...
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

(Примечание: появление строки «Инициализация сервера...» до сообщения о готовности подтверждает правильную работу нашего контекстного менеджера `lifespan` и паттерна Singleton).

4. **Внимание:** Терминал, в котором запущен Uvicorn, теперь заблокирован процессом веб-сервера. Не закрывайте это окно и не нажимайте `CTRL+C`, иначе микросервис остановится. Для выполнения последующих шагов и тестов сверните это окно терминала.

Этап 2.2. Интерактивная верификация модели и стресс-тестирование валидации типов данных через встроенную графическую песочницу Swagger UI (/docs).

Для проверки качества работы развернутой модели и верификации контура безопасности Pydantic, необходимо провести интерактивное тестирование микросервиса через автоматически сгенерированный графический интерфейс Swagger UI.

1. Откройте любой веб-браузер на вашем компьютере и перейдите в адресной строке по сетевому пути к интерактивной документации:

```
http://127.0.0.1:8000/docs
```

2. Убедитесь, что на экране успешно отобразилась веб-страница Swagger UI. Найдите в списке эндпоинтов блок **POST** `/v1/predict` (выделенный зеленым цветом) и кликните по нему, чтобы развернуть детальную спецификацию сетевого контракта.
3. Нажмите на кнопку **«Try it out»** в правой части панели. Убедитесь, что текстовое поле *Request body* стало доступным для редактирования, а внутри него автоматически появился шаблон JSON-запроса, сформированный на основе ваших примеров (`example`) из класса `IrisRequestData` .
4. Выполните **позитивное тестирование (валидный инференс)**: не меняя сгенерированные числовые значения в JSON-шаблоне, нажмите на большую синюю кнопку **«Execute»**. Прокрутите страницу чуть ниже до секции *Responses* и убедитесь, что:
    - Сервер вернул статус-код **`200`** (успешный ответ).
    - В поле *Response body* пришел корректный JSON-ответ с предсказанным индексом класса (0, 1 или 2), массивом вероятностей и системным статусом `iris_onnx_v1.0` .
5. Проведите **негативное тестирование (стресс-тест валидации данных)**. Измените значение поля `sepal_length` внутри JSON-шаблона, введя туда заведомо некорректное отрицательное число (например, `-5.1`), и нажмите кнопку **«Execute»**.
6. Изучите реакцию сервера на некорректные данные. Убедитесь, что микросервис заблокировал запрос, не допустив его до модели:
    - Сервер вернул статус-код **`422 Unprocessable Entity`** (ошибка валидации).
    - В теле ответа (*Response body*) содержится структурированное сообщение об ошибке, указывающее, что значение поля должно быть строго больше нуля: `"msg": "Input should be greater than 0"` .

7. Проведите повторный стресс-тест, передав в поле `petal_width` строковое значение в виде букв (например, `"две десятых"` вместо `0.2`), и нажмите **«Execute»**. Убедитесь, что Pydantic снова вернул ошибку `422` с сообщением о невозможности конвертации строки в число с плавающей точкой: `"msg": "Input should be a valid number, unable to parse string as a number"`.

Этап 2.3. Написание автоматического проверочного скрипта на Python (с использованием библиотеки requests или httpx) для симуляции конкурентных запросов к API.

Для автоматизации тестирования и проверки поведения микросервиса при программном обращении, необходимо написать независимый проверочный скрипт, который будет отправлять HTTP-запросы к нашему локальному API с помощью библиотеки `requests`.

1. Создайте в корневой директории вашего проекта (рядом с `main.py`) новый пустой файл с именем `client_test.py`.

2. Откройте файл client_test.py в вашей среде разработки (IDE) и добавьте в него импорт библиотеки requests для отправки сетевых запросов и модуля time для измерения задержки (Latency):

```
import time
import requests
```

3. Объявите в коде константу с URL-адресом эндпоинта инференса и сформируйте словарь с валидными тестовыми признаками, которые будут передаваться в теле POST-запроса:

```
API_URL = "http://127.0.0.1:8000/v1/predict"

# Тестовый набор признаков (эквивалентен схеме Pydantic)
payload = {
    "sepal_length": 5.1,
    "sepal_width": 3.5,
    "petal_length": 1.4,
    "petal_width": 0.2
}
```

4. Ниже по коду реализуйте функцию отправки одиночного запроса с измерением времени, за которое сервер обрабатывает транзакцию и возвращает ответ:

```
def send_request():
    start_time = time.time()

    # Отправляем POST-запрос, автоматически сериализуя словарь Python в JSON-текст
    response = requests.post(API_URL, json=payload)
```

```
    latency = (time.time() - start_time) * 1000 # Переводим секунды в
миллисекунды

    if response.status_code == 200:
        data = response.json()
        print(f"[УСПЕХ] Статус: {response.status_code} | Задержка (Latency):
{latency:.2f} мс")
        print(f"        Предсказание модели: Класс {data['predicted_class']}
| Статус: {data['model_status']}\n")
    else:
        print(f"[ОШИБКА] Статус: {response.status_code} | Ответ:
{response.text}\n")
```

5. В конце файла пропишите точку входа __main__ и симулируйте серию последовательных конкурентных запросов к серверу (например, запустив цикл из 5 итераций), чтобы оценить стабильность работы API в динамике:

```
if __name__ == "__main__":
    print("Старт автоматического тестирования ИИ-микросервиса...\n")
    for i in range(5):
        print(f"Отправка запроса №{i+1}...")
        send_request()
        time.sleep(0.5) # Небольшая пауза между запросами
```

6. Сохраните изменения в файле client_test.py. Не закрывая терминал с работающим сервером Uvicorn, откройте второе (параллельное) окно терминала, активируйте виртуальное окружение и запустите написанный тест:

```
python client_test.py
```

7. Изучите вывод в консоли. Убедитесь, что все 5 запросов завершились со статусом 200 [УСПЕХ], а среднее время задержки (Latency) не превышает нескольких десятков миллисекунд.

Задание 3: Проектирование оптимизированного многоэтапного Dockerfile

Этап 3.1. Создание базового манифеста сборки образа с разделением слоев для максимального использования кэша Docker (UnionFS) при частых изменениях исходного кода.

Для обеспечения максимальной скорости сборки контейнера при регулярных доработках кода инференса необходимо спроектировать манифест Dockerfile, который эффективно разделяет редко меняющиеся слои (зависимости) и часто меняющиеся слои (исходный код).

1. Инициализируйте среду Docker на вашем ПК. Если Docker не был установлен ранее, разверните его в соответствии с вашей операционной системой:

- **Linux:**

  Выполните в терминале стандартные команды установки:

  ```
  sudo apt update && sudo apt install docker.io -y
  ```

- **Windows / macOS:**

  Скачайте и установите официальный графический пакет **Docker Desktop** с

сайта docker.com (для Windows убедитесь, что в процессе установки активирован бэкенд WSL 2).

2. Запустите приложение Docker Desktop (или службу Docker в Linux через команду `sudo systemctl start docker`).

3. Откройте терминал и выполните проверочную команду для верификации успешного отклика движка:

```
docker version
```

(Системная справка: Если команда возвращает ошибку доступа в Linux, добавьте текущего пользователя в системную группу docker: `sudo usermod -aG docker $USER`*, после чего полностью перезапустите сессию терминала).*

4. Перед описанием Dockerfile зафиксируйте все текущие библиотеки вашего проекта. Откройте терминал в корневой папке (где активировано виртуальное окружение `venv`) и выгрузите точный список зависимостей в текстовый файл:

```
pip freeze > requirements.txt
```

(Внимание: убедитесь, что в файле requirements.txt находятся только необходимые для работы сервера библиотеки, такие как fastapi, uvicorn, onnxruntime и т.д., без лишнего мусора из глобальной системы).

5. Откройте вашу среду разработки (IDE) в корне проекта и создайте новый пустой файл с именем Dockerfile (строго с заглавной буквы и обязательно без какого-либо расширения вроде .txt).

6. Задайте базовую операционную систему с предустановленным интерпретатором Python, используя максимально компактный официальный образ -slim (на базе урезанного дистрибутива Debian), и укажите рабочую директорию внутри контейнера:

```
FROM python:3.10-slim
WORKDIR /app
```

7. Опишите слой копирования файла зависимостей. Переносите строго только один файл requirements.txt, а не весь проект целиком, чтобы не сбрасывать кэш Docker при правках кода:

```
COPY requirements.txt .
```

8. Пропишите инструкцию установки библиотек. Обязательно используйте флаг --no-cache-dir, чтобы бинарные архивы пакетов удалялись сразу после установки и не запекались мертвым грузом внутри текущего слоя UnionFS:

```
RUN pip install --no-cache-dir -r requirements.txt
```

9. Разместите инструкции копирования самого приложения в самом конце манифеста. Перенесите бинарный граф весов модели iris_model.onnx и файлы серверного кода main.py (теперь при любых изменениях в коде Docker будет пересобирать только этот

финальный легкий слой, извлекая тяжелый слой pip install из готового локального кэша за 0 секунд):

```
COPY iris_model.onnx .
COPY main.py .
```

10. Задайте команду по умолчанию для запуска ASGI-сервера Uvicorn при старте контейнера, указав прослушивание всех сетевых интерфейсов (0.0.0.0) и стандартный внутренний порт 8000:

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

11. Сохраните изменения в файле Dockerfile. Убедитесь, что получился связный базовый манифест из 7 строк, готовый к дальнейшей многоэтапной (Multi-stage) модернизации.

Этап 3.2. Интеграция технологии многоэтапной сборки (Multi-stage builds) с выделением стадий builder (тяжелая среда компиляции) и production (минималистичный чистый образ на базе -slim дистрибутива).

Для радикального уменьшения физического размера продакшен-образа и очистки системы от избыточных инструментов сборки, необходимо перепроектировать созданный манифест, применив технологию многоэтапной сборки (**Multi-stage builds**).

1. Откройте ранее созданный файл `Dockerfile` в вашей среде разработки (IDE). Мы модернизируем его структуру, разделив один линейный процесс на две изолированные стадии сборки.

2. Опишите первый, изолированный этап – стадию компиляции (назовем её **builder**). На этом этапе мы берем базовый образ и доустанавливаем в него системные компиляторы, необходимые для сборки некоторых зависимостей:

```
# === ЭТАП 1: Сборщик (Builder) ===
FROM python:3.10-slim AS builder
WORKDIR /build

# Устанавливаем системные инструменты компиляции (gcc, g++ и др.)
RUN apt-get update && apt-get install -y --no-install-recommends build-
essential \
    && rm -rf /var/lib/apt/lists/*
```

3. В рамках этого же первого этапа скопируйте файл `requirements.txt` и запустите команду компиляции пакетов в готовые бинарные архивы (**wheels**), сохранив их в локальную временную директорию `/build/wheels`:

```
COPY requirements.txt .
RUN pip wheel --no-cache-dir --wheel-dir=/build/wheels -r requirements.txt
```

(Системная справка: на этом этапе все библиотеки скачиваются и при необходимости компилируются с привлечением build-essential. Весь этот тяжелый мусор останется внутри изолированного слоя builder и не попадет в финальный продукт).

4. Ниже в этом же файле начните описание второго, финального этапа сборки (стадии **production**). Для этого заново вызовите директиву `FROM`, инициализировав кристально чистый и легковесный образ, лишенный компиляторов:

```
# === ЭТАП 2: Финальный продакшен-образ ===
FROM python:3.10-slim
WORKDIR /app
```

5. Примените специальную директиву `COPY --from`, чтобы физически перенести результаты сборки (папку с wheels) и сам файл зависимостей из хранилища первого этапа во второй, чистый контур:

```
# Извлекаем готовые скомпилированные библиотеки и манифест, игнорируя
мусор сборщика
COPY --from=builder /build/wheels /wheels
COPY --from=builder /build/requirements.txt .
```

6. Запустите установку пакетов во втором контейнере. Используйте флаги `--no-index` и `--find-links`, чтобы пакетный менеджер `pip` установил библиотеки строго из локально скопированных бинарных файлов без обращения к интернету, после чего сразу удалите временную папку `/wheels` для экономии места:

```
RUN pip install --no-cache-dir --no-index --find-links=/wheels -r
requirements.txt \
    && rm -rf /wheels
```

7. Завершите написание многоэтапного `Dockerfile`, скопировав файлы вашего интеллектуального приложения и прописав стандартную команду запуска сервера Uvicorn (обратите внимание: файл модели копируется перед кодом, чтобы оптимизировать кэширование при частых правках логики API):

```
COPY iris_model.onnx .
COPY main.py .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8. Сохраните изменения в файле `Dockerfile`. Убедитесь, что ваш итоговый манифест содержит два блока `FROM` и логически разделен на стадию сборки тяжелых зависимостей и стадию доставки легкого кода.

Этап 3.3. Использование флагов безопасности и предотвращения раздувания контейнера (отсечение кэша пакетных менеджеров --no-cache-dir, очистка временных системных списков).

Для исключения скрытого раздувания Docker-контекста и образов временными файлами Git, локальными базами данных и кэшем Python, необходимо настроить контур фильтрации проекта с помощью файла `.dockerignore` и верифицировать флаги очистки в манифесте сборки.

1. Откройте вашу среду разработки (IDE) в корневой директории проекта и создайте новый пустой файл с именем `.dockerignore` (строго с точки в начале, по аналогии с `.gitignore`).

2. Занесите в созданный файл `.dockerignore` перечень всех локальных директорий и служебных файлов, которые категорически запрещено отправлять в контекст сборки Docker:

```
venv/
.git/
.gitignore
.dvc/
.dvcignore
data/
client_test.py
mlflow.db
mlflow_artifacts/
__pycache__/
*.pyc
.DS_Store
```

(Системная справка: при запуске команды docker build вся корневая папка проекта упаковывается и передается демону Docker в качестве контекста сборки. Если не создать .dockerignore, тяжелая папка виртуального окружения venv/ и гигабайты скрытой истории .git/ будут каждый раз копироваться во временный буфер Docker, сильно замедляя старт сборки и создавая риски случайного запекания мусора в образ через директивы общего копирования).

3. Откройте ваш `Dockerfile` и проверьте строки установки системных пакетов в блоке **builder**. Убедитесь, что команда обновления индексов `apt-get update` и установка компиляторов `apt-get install` объединены в одну общую инструкцию через оператор `&&`, а в самом концу строки прописана принудительная очистка временных кэш-списков Linux:

```
RUN apt-get update && apt-get install -y --no-install-recommends build-
essential \
    && rm -rf /var/lib/apt/lists/*
```

Запомните инженерное правило: если разбить эти команды на две отдельные директивы (`RUN apt-get update` и отдельная `RUN apt-get install...`), то скачанные из интернета тяжелые списки репозиториев Linux намертво запекутся в промежуточном слое UnionFS. Последующая команда `rm -rf` в новом слое уже не сможет уменьшить физический размер итогового образа на диске.

4. Проверьте все вызовы пакетного менеджера `pip` в коде вашего `Dockerfile` (как на этапе **builder**, так и на этапе **production**). Убедитесь, что каждая команда содержит

в себе флаг `--no-cache-dir`, а на финальном этапе развертывание идет строго из локальной папки:

```
# На этапе builder:
RUN pip wheel --no-cache-dir --wheel-dir=/build/wheels -r requirements.txt

# На этапе production:
RUN pip install --no-cache-dir --no-index --find-links=/wheels -r
requirements.txt \
    && rm -rf /wheels
```

(Флаг --no-cache-dir приказывает пакетному менеджеру разворачивать библиотеки напрямую в целевые системные каталоги Python, полностью блокируя сохранение скачанных .tar.gz и .whl архивов во временную скрытую папку ~/.cache/pip, что экономит до 50% объема слоя).

5. Сохраните изменения в файлах `.dockerignore` и `Dockerfile`. Проект полностью очищен от мусора и готов к безопасной промышленной сборке.

Задание 4: Сборка, запуск и аудит Docker-контейнера

Этап 4.1. Выполнение CLI-команд сборки локального образа с присвоением версионных тегов. Аудит физического размера получившегося слепка системы.

Для трансляции написанного манифеста в готовый к развертыванию продукт необходимо запустить процесс компиляции Docker-образа и провести инженерный аудит его итогового физического размера на жестком диске.

1. Откройте терминал командной строки и убедитесь, что вы находитесь в корневой директории вашего проекта, где физически лежат файлы `Dockerfile`, `.dockerignore` и `main.py`.

2. Запустите процесс многоэтапной сборки вашего локального Docker-образа, передав команду docker build с флагом -t для присвоения логического имени (тега) и версионной метки в формате имя_репозитория:версия:

```
docker build -t iris_api_service:v1.0 .
```

(Внимание: точка в самом конце команды обязательна – она указывает Docker-демону, что контекст сборки и сам файл Dockerfile находятся в текущем каталоге).

3. Внимательно следите за выводом логов сборки в терминале. Обратите внимание, как Docker последовательно выполняет шаги: сначала разворачивается стадия [builder], где компилируются пакеты wheels, а затем управление переходит к стадии [production], куда копируются только чистые бинарники. По окончании сборки вы должны увидеть финальную строку:

```
=> => naming to docker.io/library/iris_api_service:v1.0
```

4. Проведите локальный аудит созданного слепка системы. Запросите у Docker CLI список всех доступных локальных образов и найдите среди них собранный сервис:

```
docker images
```

5. Изучите сгенерированную аналитическую таблицу в консоли. Найдите строку с REPOSITORY: iris_api_service и TAG: v1.0. Обратите внимание на крайнюю правую колонку SIZE (Размер).

6. Зафиксируйте итоговый вес вашего образа в мегабайтах (благодаря многоэтапной сборке, дистрибутиву -slim и очистке кэша, размер контейнера со встроенным веб-сервером и рантаймом ИИ-модели должен составить всего порядка 150–250 МБ, вместо нескольких гигабайт).

Этап 4.2. Запуск изолированного контейнера в фоновом режиме с пробросом сетевых портов (-p) из контейнера на хост-машину.

Для развертывания собранного интеллектуального микросервиса необходимо запустить скомпилированный образ в виде изолированного фонового процесса, связав внутренний сетевой порт Docker-контейнера с реальным портом вашей хост-машины.

1. Убедитесь, что ваш ранее запущенный локальный сервер Uvicorn (из Задания 2) остановлен, чтобы избежать конфликта сетевых портов. Если он все еще работает, перейдите в то окно терминала и нажмите комбинацию клавиш `CTRL+C` для освобождения порта `8000`.

2. Запустите изолированный Docker-контейнер из созданного образа iris_api_service:v1.0, объединив в команде три ключевых флага автоматизации:

```
docker run -d -p 8000:8000 --name iris_app_container iris_api_service:v1.0
```

Разбор архитектурных флагов команды:

- `-d` (*detached mode*) – приказывает Docker запустить контейнер в фоновом режиме в качестве демона. Команда мгновенно вернет управление терминалом пользователю, а сам микросервис останется работать автономно.
- `-p 8000:8000` (*port mapping*) – прорубает сетевой мост сквозь барьер изоляции. Первая цифра – это порт на вашей реальной хост-машине (компьютере), вторая цифра – внутренний порт внутри Docker-контейнера, который мы прописали в инструкции `CMD` нашего `Dockerfile`.
- `--name iris_app_container` – присваивает запущенному вычислительному процессу понятное системное имя для удобства последующего администрирования.

3. Проверьте статус запущенного процесса в операционной системе хоста. Запросите у Docker CLI список всех активных контейнеров:

```
docker ps
```

4. Изучите сгенерированную таблицу в консоли. Убедитесь, что контейнер с именем iris_app_container имеет статус Up (Работает), а в колонке PORTS присутствует перенаправление трафика: 0.0.0.0:8000->8000/tcp.

5. Проведите аудит внутренних системных логов запущенной нейросети. Проверьте, что внутри изолированной среды Uvicorn успешно инициализировал и прогрел модель ONNX через lifespan:

```
docker logs iris_app_container
```

В выводе терминала вы должны увидеть те же строки стартапа, что и при локальном запуске, но теперь они выполняются внутри изолированной файловой системы контейнера.

Этап 4.3. Финальная проверка работоспособности контейнеризированного интеллектуального приложения через утилиту Postman или Swagger UI, развернутый внутри изолированной среды.

Для проведения финальной приемки и верификации развернутого интеллектуального микросервиса необходимо протестировать эндпоинты через внешние инструменты (Swagger UI или Postman), чтобы доказать успешный проброс трафика сквозь контур контейнеризации.

1. Откройте любой веб-браузер на вашей хост-машине и перейдите по адресу интерактивной документации Swagger UI, которая теперь генерируется и отдается напрямую изнутри изолированного Docker-контейнера:

```
http://127.0.0.1:8000/docs
```

2. Если вы используете для тестирования утилиту Postman, запустите приложение Postman на вашем компьютере и настройте новую вкладку запроса со следующими параметрами:

- **Метод запроса:** Выберите `POST` в выпадающем списке.
- **Адрес запроса (URL):** Введите `http://127.0.0.1:8000/v1/predict`.
- **Вкладка Headers:** Убедитесь в наличии заголовка `Content-Type: application/json`.
- **Вкладка Body:** Выберите режим `raw`, установите тип `JSON` и вставьте в текстовое поле валидный пакет признаков:

```
{
  "sepal_length": 6.3,
  "sepal_width": 2.5,
  "petal_length": 5.0,
  "petal_width": 1.9
}
```

3. Отправьте запрос на сервер (нажав синюю кнопку «Execute» в Swagger UI или кнопку «Send» в Postman).

4. Проведите инженерный аудит полученного ответа. Убедитесь, что:

- Статус-код ответа равен **`200 OK`**.
- Время ответа (*Time / Latency*) измеряется миллисекундами, что доказывает отсутствие дисковых оверхедов и нативную скорость работы ONNX Runtime в контейнере.

- Тело JSON-ответа содержит корректную структуру `predicted_class`, `probabilities` и `model_status`.

5. Проверьте фиксацию транзакции на стороне сервера. Вернитесь в терминал вашей командной строки и повторно запросите логи контейнера:

```
docker logs iris_app_container
```

Убедитесь, что в самом конце логов утилиты Uvicorn появилась новая строка, фиксирующая ваш входящий POST-запрос и факт отправки ответа `200`:

```
INFO:     172.17.0.1:xxxxx - "POST /v1/predict HTTP/1.1" 200 OK
```

(Системная справка: сетевой адрес `172.17.0.1` – это стандартный виртуальный шлюз внутренней подсети Docker-моста, через который хост-система общается с изолированной средой).

6. Поздравляем, ваше интеллектуальное приложение полностью контейнеризировано, оптимизировано и готово к промышленному деплою в облачные кластеры! Очистите локальное окружение, остановив и удалив тестовый контейнер из системы:

```
docker stop iris_app_container
docker rm iris_app_container
```

4. Требования к содержанию и оформлению отчета

Структура титульного листа и вводной части

Отчет по лабораторной работе №2 оформляется в электронном виде в формате PDF и должен строго соответствовать академическим правилам документирования программных систем. Объем отчета – не более 10–12 страниц.

1. **Титульный лист:** Оформляется по стандартному шаблону кафедры. Обязательные реквизиты:
   - Полное наименование университета, факультета и выпускающей кафедры.
   - Название дисциплины: «Основы развертывания интеллектуальных приложений».
   - Тему лабораторной работы №2: «Разработка API и контейнеризация модели».
   - Сведения о студенте (ФИО, учебная группа) и проверяющем преподавателе (ФИО, ученая степень/должность).
   - Город и 2026 год выполнения.
2. **Вводная часть:** Начинается со второй страницы отчета и содержит:
   - **Цель работы:** Дословная формулировка из подраздела 1.1 данных методических указаний.

- **Задачи работы:** Перечень четырех базовых заданий (разработка API, валидация, написание многоэтапного Dockerfile, локальное тестирование контейнера).
- **Описание используемой модели:** Краткое текстовое указание, на какую бизнес-задачу и архитектуру модели (из Лабораторной работы №1) был разработан данный микросервис.

Требования к фиксации результатов (скриншоты, листинги кода, логи)

Основная часть отчета должна поэтапно и наглядно иллюстрировать ход выполнения работы с обязательными доказательствами корректности функционирования как самого кода FastAPI, так и изолированного Docker-контейнера.

- **Требования к скриншотам:**
  - Все скриншоты терминала и графических интерфейсов должны быть высокого разрешения и читаемости. Смазанные или обрезанные изображения не принимаются к оцениванию.
  - *Обязательный скриншот №1:* Окно браузера с развернутым веб-интерфейсом **Swagger UI** по адресу `http://127.0.0.1:8000/docs`, где видна структура сгенерированных Pydantic-моделей для `IrisRequestData` и `IrisResponseData`.
  - *Обязательный скриншот №2:* Успешное **позитивное тестирование** в Swagger UI: статус-код `200` и корректное тело JSON-ответа от модели.
  - *Обязательный скриншот №3:* Результат **негативного стресс-тестирования** в Swagger UI при вводе отрицательного значения признака: статус-код `422` и текстовое описание ошибки от Pydantic.
  - *Обязательный скриншот №4:* Вывод логов терминала при успешном выполнении команды сборки оптимизированного многоэтапного образа: `docker build -t iris_api_service:v1.0 .`.
  - *Обязательный скриншот №5:* Вывод команды `docker images` в консоли, где четко виден итоговый физический **размер (Size)** созданного образа в мегабайтах (для контроля качества оптимизации).
  - *Обязательный скриншот №6:* Результат финального инференса контейнеризированной модели через утилиту **Postman** (POST-запрос на порт `8000`), подтверждающий сквозной проброс трафика.
- **Requirements к листингам кода:**

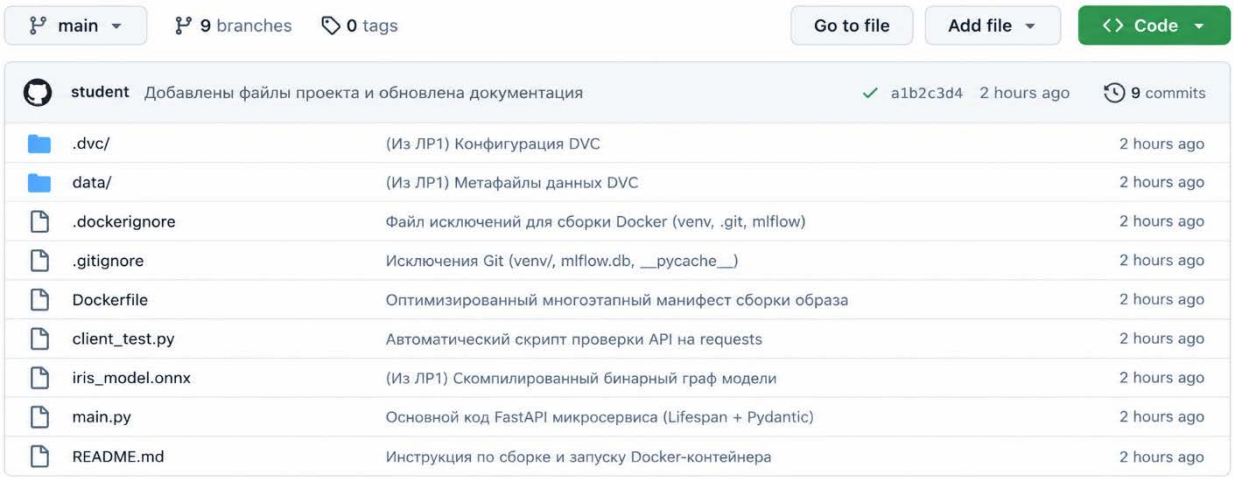
- В отчет вставляются полные, структурированные исходные тексты файлов `main.py` (FastAPI-сервер), `client_test.py` (автоматический клиент) и `Dockerfile` (многоэтапный манифест сборки).
- Код должен быть оформлен стандартным моноширинным шрифтом с подсветкой синтаксиса или в виде текстовых блоков «Код». **Вставка кода скриншотами запрещена.**

- **Требования к логам:**
  - В отчет копируется текстовый лог команды `docker logs iris_app_container`, содержащий строки успешной отработки `lifespan` (загрузка весов ONNX при старте сервера) и входящую строку обращения от Postman со статус-кодом `200 OK`.

Формат представления кода (ссылка на репозиторий)

Проверка лабораторной работы базируется на аудите живой кодовой базы и истории Git-коммитов студента.

1. Все созданные скрипты и конфигурационные файлы должны быть закоммичены и выложены в тот же публичный репозиторий на **GitHub** или **GitLab**, который использовался в Лабораторной работе №1. Ссылка на репозиторий дублируется во вводной части отчета.
2. Каждое выполненное задание должно сопровождаться понятным комментарием в истории коммитов (например, `feat: add lifespan singleton for onnx`, `build: implement multi-stage dockerfile`).
3. **Итоговая структура репозитория в Git:**



Критическое требование: Убедитесь, что файл `.dockerignore` настроен верно. Папка виртуального окружения `venv/`, локальный кэш артефактов `mlflow_artifacts/` и скрытый каталог `.git/` **не должны** физически попадать внутрь Docker-образа при сборке.

5. Контрольные вопросы для защиты лабораторной работы

Архитектура API и валидация данных (FastAPI и Pydantic)

1. Опишите физический смысл и этапы цикла «Запрос – Ответ» (Request – Response) при инференсе модели машинного обучения по протоколу HTTP REST. Почему для отправки векторов признаков или изображений архитектурно необходимо использовать метод `POST`, а не `GET`?
2. Раскройте суть паттерна проектирования `Stateless` (без сохранения состояния) в контексте разработки ИИ-микросервисов. Каким образом соблюдение этого паттерна влияет на возможности горизонтального масштабирования интеллектуального приложения в кластерах Kubernetes?
3. В чем заключается критическая опасность архитектурного антипаттерна, при котором загрузка и инициализация весов модели (например, вызов `mlflow.sklearn.load_model` или `ort.InferenceSession`) происходят непосредственно внутри тела функции-эндпоинта? К каким последствиям для метрик `Latency` и `Memory Footprint` это приведет под нагрузкой?
4. Объясните, как паттерн `Singleton` (Одиночка) решает проблему нерационального использования оперативной памяти сервера. Опишите механизм работы менеджера контекста `lifespan` в FastAPI: в какие моменты времени выполняются блоки кода до и после оператора `yield`?
5. Какую роль играет библиотека `Pydantic` в контуре безопасности и стабильности ИИ-микросервиса? Каким образом декларативное описание схем данных и строгая типизация защищают низкоуровневый C++ граф вычислений модели от критических ошибок сегментации памяти (`Segmentation Fault`)?

Асинхронность и конкурентный инференс

1. Объясните принцип работы цикла событий (`Event Loop`) в асинхронной архитектуре AsyncIO/FastAPI. В чем заключается фундаментальное различие между задачами ввода-вывода (`I/O-bound`) и вычислительными задачами (`CPU-bound`)?

2. К какому инфраструктурному коллапсу приведет объявление эндпоинта инференса тяжелой нейросети через ключевые слова `async def predict(...)`? Почему чистые математические вычисления намертво заклинивают Event Loop и как это отражается на способности сервера принимать запросы от других параллельных пользователей?
3. Опишите внутреннюю механику поведения FastAPI, когда эндпоинт инференса объявляется как стандартная синхронная функция `def predict(...)`. Куда сервер принудительно перенаправляет этот вызов и как это позволяет основному потоку Event Loop оставаться свободным?
4. С какими рисками и техническими проблемами сталкивается инженер при организации конкурентного (параллельного) доступа множества сетевых потоков к единому глобальному объекту модели в оперативной памяти? Что такое состояние гонки (`Race Condition`) применительно к вызову метода `.predict()`?
5. В каких сценариях разработки интеллектуальных систем в FastAPI действительно жизненно необходимо объявлять эндпоинты как асинхронные (`async def`)? Приведите пример архитектурной цепочки, где асинхронность дает максимальный выигрыш в производительности.

Контейнеризация и оптимизация образов (Docker)

1. Сформулируйте концептуальные отличия контейнеризации на базе Docker от классической виртуализации (виртуальных машин). За счет каких встроенных механизмов ядра Linux (`Namespaces` и `Cgroups`) достигается нативная скорость выполнения вычислений в контейнере без развертывания гостевых операционных систем?
2. Раскройте устройство слоистой архитектуры файловой системы `UnionFS`. Почему порядок написания директив в `Dockerfile` жестко влияет на скорость повторной сборки образа (CI/CD пайплайна)? Опишите паттерн разделения копирования зависимостей (`requirements.txt`) и исходного кода.
3. С какой целью при установке пакетов через `pip install` или `apt-get` внутри Dockerfile необходимо принудительно использовать флаги `--no-cache-dir` и команды очистки системных списков вроде `rm -rf /var/lib/apt/lists/*`? К какому раздуванию образа приводит игнорирование этих флагов?

4. Объясните технологию многоэтапной сборки (`Multi-stage builds`). За счет чего использование нескольких директив `FROM` и команды `COPY --from` позволяет избавиться от тяжелых системных компиляторов (`build-essential`, `gcc`) и минимизировать размер финального продакшен-образа?
5. Какую роль играет файл `.dockerignore` на этапе выполнения команды `docker build`? Опишите последствия для безопасности и производительности ИТ-системы, если папка локального виртуального окружения `venv/` или скрытый каталог репозитория `.git/` не будут занесены в файл исключений Docker.